

BLEI-E MULTI-ASSET RISK ENGINE

Prototype Application Specification **v1.0**

Date	June 23, 2026
Author	Dashawn Ramel Bledsoe drb-apes
GitHub	github.com/drb-apes
Repository	github.com/drb-apes/institutional-grade-nil-valuation-reporting-and-exchange
Whitepaper	docs.google.com/document/d/1zQRohQ7oBBscN9X-WRQlrA6FIAQYZxDP/edit
Document Type	Prototype Application Specification
Target Audience	Senior Engineers, Quant Developers, Product Managers

CONFIDENTIAL — INTERNAL USE ONLY

This document contains proprietary and confidential information belonging to Dashawn Ramel Bledsoe / drb-apes.

Unauthorized disclosure, reproduction, or distribution is strictly prohibited.

© 2026 Dashawn Ramel Bledsoe. All rights reserved.

Table of Contents

Section 1 — Product Overview

- 1.1 Purpose
- 1.2 Target Users
- 1.3 Core Value Proposition
- 1.4 Platform Scope — Version 1.0

Section 2 — Application Screens & UI Specification

- 2.1 Screen 1 — Login & Authentication
- 2.2 Screen 2 — Main Dashboard (Portfolio Manager View)
- 2.3 Screen 3 — Athlete Deep Dive
- 2.4 Screen 4 — Factor Model Workbench (Quant Analyst View)
- 2.5 Screen 5 — Risk Officer Dashboard
- 2.6 Screen 6 — Nielsen Sports Intelligence Feed
- 2.7 Screen 7 — Compliance & Governance
- 2.8 Screen 8 — Settings & API Configuration

Section 3 — Data Architecture

- 3.1 Signal Ingestion Pipeline
- 3.2 Real-Time Streaming Architecture
- 3.3 Factor Calculation
- 3.4 Database Schema (Key Tables)

Section 4 — API Specification

- 4.1 REST API Endpoints (FastAPI)
- 4.2 Authentication Headers & Rate Limiting

Section 5 — Technology Stack Summary

Section 6 — Development Roadmap

- 6.1 Version 1.0 — Q3 2026

6.2 Version 1.1 — Q4 2026

6.3 Version 2.0 — Q1 2027

Section 7 — Repository & Reference Links

CONFIDENTIAL | BLEI-E App Spec v1.0 | drb-apes | June 2026

Section 1 — Product Overview

1.1 Purpose

BLEI-E (Biometric-Linked Equity Intelligence Engine) is a real-time, multi-asset risk intelligence platform that converts athlete biometric, behavioral, and virality signals into institutional-grade equity risk factors and portfolio overlays. The platform operationalizes the BLEI-E five-factor model — comprising Performance Momentum Factor-E (PMF-E), Sentiment & Exposure Factor-E (SEF-E), Viral Virality Engine-E (VVE-E), Fan Base Resonance-E (FBR-E), and Clutch-State Encoding-E (CSE-E) — into a structured Composite Risk Score (CRS) that drives portfolio-level signals and overlay recommendations.

This document specifies the complete architecture, screens, data flows, API contracts, and user interaction models for the BLEI-E prototype application — **Version 1.0**. It is written for senior engineers, quantitative developers, and product managers operating in institutional financial environments including banks, hedge funds, and quantitative asset managers. All specifications are developer-ready and intended to drive direct implementation.

Scope Note

Version 1.0 covers the NBA athlete universe only. NFL and EPL expansion is scoped to Version 2.0 (Q1 2027). All equity exposure data in v1.0 is anchored to Madison Square Garden Sports Corp. (MSGs) and Madison Square

Garden Entertainment Corp. (MSGE) as primary franchise proxy equities.

1.2 Target Users

BLEI-E v1.0 is designed for five distinct user roles, each mapped to a primary application screen and RBAC permission tier:

Role	Primary Responsibility	Primary Screen(s)	RBAC Tier
Quant Analyst	Factor model calibration, back-test runner, signal validation	Factor Model Workbench (Screen 4)	ANALYST
Portfolio Manager	Real-time CRS dashboard, overlay recommendations, position sizing	Main Dashboard (Screen 2), Athlete Deep Dive (Screen 3)	PM
Risk Officer	Risk state monitoring, stress testing, audit trail oversight	Risk Officer Dashboard (Screen 5)	RISK
Data Engineer	Signal ingestion pipelines, API configuration, data quality monitoring	Settings & API Configuration (Screen 8)	ENGINEER
Compliance Officer	Governance dashboard, audit logs, HIPAA/GDPR data handling oversight	Compliance & Governance (Screen 7)	COMPLIANCE

1.3 Core Value Proposition

BLEI-E Pre-Reactive Signal Thesis

BLEI-E signals precede market repricing by 2–3 trading days. The platform converts real-time physiological data into structured equity signals before sell-side analysts react. Competitive edge: sub-200ms signal latency from athlete event to CRS update to portfolio overlay recommendation.

Traditional sports-equity adjacency analysis relies on retrospective media coverage, earnings calls, and lagging sentiment surveys. BLEI-E inverts this model by ingesting athlete biometric telemetry — including Heart Rate Variability (HRV RMSSD), VO₂ max estimates, session load from Catapult Vector 8 hardware, and WHOOP recovery scores — at the source, before outcomes manifest in public market data. The resulting Composite Risk Score (CRS, 0–4 state scale) drives deterministic overlay recommendations to portfolio management workflows.

The platform's quantitative backbone is grounded in academic methodology: Fama-MacBeth two-pass regression with Newey-West HAC standard errors validates factor premia; GRS joint alpha testing confirms multi-factor explanatory power; and look-ahead bias controls enforce a minimum 24-hour signal-to-price lag via `merge_asof` join semantics.

CRS State Definition Table

CRS State	Label	Color Code	Interpretation	Default Action
0	Baseline	GREY	No elevated risk signals. All factors within 1 standard deviation of baseline.	HOLD
1	Watch	YELLOW	One factor elevated >1.5σ from baseline. Monitor closely.	REVIEW
2	Elevated	ORANGE	Two or more factors	HEDGE / REDUCE

CRS State	Label	Color Code	Interpretation	Default Action
			elevated, or one factor $>2\sigma$. Overlay trigger zone.	
3	High Risk	RED	CRS composite $>2.5\sigma$ from seasonal mean. Active overlay recommended.	SHORT / HEDGE
4	Critical	CRITICAL RED PULSE	All five factors elevated simultaneously or extreme single-factor event. Maximum alert.	URGENT — ALERT PM

1.4 Platform Scope — Version 1.0

The BLEI-E v1.0 prototype is a full-stack web application built on the following technology pillars:

- **Web Application:** React 18 + TypeScript frontend, served as single-page application (SPA)
- **Backend:** Python 3.12 FastAPI microservices — async-native, OpenAPI 3.1 auto-generated docs
- **Risk Calculation Core:** C++ compiled module (latency-critical factor normalization and CRS aggregation), called via Python `ctypes`
- **Database:** DuckDB analytics layer on year-partitioned Apache Parquet files — no OLTP database in v1.0
- **Streaming:** WebSocket real-time CRS push; Redis pub/sub inter-service event queue
- **Auth:** OAuth 2.0 PKCE flow + Role-Based Access Control (RBAC); JWT tokens in httpOnly cookies
- **Deployment:** Docker containers, Kubernetes orchestration (cloud-agnostic)

- **Monitoring:** Prometheus metrics collection + Grafana dashboards
- **CI/CD:** GitHub Actions (consistent with open-source drb-apes repository)

CONFIDENTIAL | BLEI-E App Spec v1.0 | drb-apes | June 2026

Section 2 — Application Screens & User Interface Specification

This section defines all eight application screens in the BLEI-E v1.0 prototype. Each specification includes a functional description, panel/layout structure, element inventory, behavioral rules, and technical implementation notes. All screens enforce authentication and RBAC before rendering. Unauthorized role access redirects to a 403 error page with compliance-logged event.

UI Framework Note

The frontend is React 18 + TypeScript, styled with Tailwind CSS utility classes. Real-time data elements use Zustand state management. Charts are rendered with Recharts (declarative) and D3.js (custom overlays). All chart data is sourced from FastAPI REST endpoints or WebSocket streams — no client-side data fabrication.

2.1 Screen 1 — Login & Authentication

SCREEN 1 | LOGIN & AUTHENTICATION | All Roles

Description

Secure institutional login portal. Entry point for all BLEI-E users. Enforces MFA on every login session regardless of role. No persistent local sessions — every session requires fresh authentication after token expiry.

UI Elements

- Institution logo placeholder (Morgan Stanley / client institution branding slot — configurable via Settings)
- **Email field:** institutional domain validation enforced (e.g., @morganstanley.com, @citadel.com); rejects personal email domains at client-side
- **Password field:** masked input, minimum 12 characters, complexity enforced
- **MFA prompt:** TOTP (Google Authenticator / Authy) or hardware security key (FIDO2/WebAuthn)
- **SSO button:** "Sign in with Institutional SSO" — SAML 2.0 federation endpoint (configurable per client deployment)
- Forgot password link → triggers compliance-logged reset email, no self-service password reset

Behavior Rules

- On successful auth: route user to their role-default dashboard screen
- Failed auth: 3-attempt lockout; lockout event auto-logged to `audit_log` table with IP address and timestamp
- Lockout alert: pushed to Compliance Officer inbox (in-app + email)
- Role assignment: read from identity provider on JWT issuance — cannot be self-modified by user
- Session idle timeout: 30 minutes (configurable per institution); prompts re-auth modal before logout

Technical Notes

- OAuth 2.0 PKCE flow — authorization code never exposed in URL fragment
- JWT access token: 15-minute expiry
- Refresh token: 24-hour expiry, rotated on each use
- All tokens stored in httpOnly cookies — no localStorage or sessionStorage
- CSRF protection: SameSite=Strict cookie attribute + CSRF token header on all state-mutating requests
- TLS 1.3 required on all connections; TLS 1.2 allowed for legacy institutional proxies

2.2 Screen 2 — Main Dashboard (Portfolio Manager View)

SCREEN 2 | MAIN DASHBOARD | Role: PM, Risk Officer, Analyst (read-only overlay panel)

Description

Real-time command center for CRS states, five-factor signals, and overlay recommendations across all tracked athletes. Three-panel layout designed for dense institutional trading desk usage. WebSocket-powered with zero-refresh live data.

Three-Panel Layout Overview

Panel	Position	Primary Function	Data Source
Athlete Watchlist	Left (~25% width)	Scrollable athlete roster with CRS badges	GET /api/v1/athletes

Panel	Position	Primary Function	Data Source
Live CRS Feed	Center (~50% width)	Real-time CRS ticker and sparklines	WS /ws/crs
Overlay Recommendations	Right (~25% width)	Active overlay actions by urgency	GET /api/v1/athletes/{hash}/overlays

Left Panel — Athlete Watchlist

- Scrollable list of tracked athletes: name, sport, team, current CRS state badge (color-coded 0-4 per state table in Section 1.3)
- Search/filter: by sport, team, CRS state, factor name
- Sort by: CRS descending, PMF-E, SEF-E, VVE-E, FBR-E, CSE-E (ascending/descending toggle)
- Add/remove athlete button — role-restricted to PM and Engineer roles
- Pin athlete to top of list (persisted in user preferences)

Center Panel — Live CRS Feed

- Real-time CRS ticker: athlete name, current CRS value (0-4), delta from last update (+/- with directional arrow), last update timestamp (UTC, millisecond precision)
- WebSocket-powered — updates push automatically without page refresh; reconnect logic with exponential backoff on disconnect
- System-wide risk state banner at panel top: displays current highest CRS state across all tracked athletes, color-coded
- 24-hour CRS history sparkline per athlete row (Recharts LineChart, no axes, visual trend only)
- Click any athlete row → expands to full five-factor breakdown inline; secondary click → navigates to Athlete Deep Dive (Screen 3)
- Stale data indicator: if WebSocket connection lost >5 seconds, all CRS values display with orange stale badge

Right Panel — Overlay Recommendation Engine

- Active overlay recommendations listed in urgency order (CRS State 4 at top)
- Each recommendation card displays: athlete name, trigger factor(s), recommended action (LONG / SHORT / HEDGE / HOLD), target instrument (ticker or derivative), confidence level (%)
- **"Send to OMS" button:** Order Management System integration placeholder — disabled in v1.0, live in v2.0. Button visible but labeled "OMS Integration — v2.0"
- Dismiss button: removes from active queue, logs to `audit_log` with timestamp and user ID
- Acknowledge button: marks as reviewed, sends compliance timestamp, leaves in log view
- Acknowledged-by field visible to Risk Officer and Compliance roles

Bottom Status Bar

- Signal ingestion health indicator: green/yellow/red per data source
- API latency display: current p95 latency ms for each connected API
- Last Gracenote push timestamp
- WebSocket connection status: CONNECTED (green) / RECONNECTING (orange) / DISCONNECTED (red)
- Active user count on current session (Risk Officer visibility only)

Technical Notes

- WebSocket endpoint: `wss://api.blei-e.io/ws/crs` (production);
`ws://localhost:8000/ws/crs` (dev)
- State management: Zustand store — `useCRSStore`, `useAthleteStore`, `useOverlayStore`
- Sparkline data: pre-fetched on dashboard mount via `GET /api/v1/athletes/{hash}/factors?window=24h`

- Overlay recommendations re-fetched every 60 seconds as fallback if WebSocket push missed

2.3 Screen 3 — Athlete Deep Dive

SCREEN 3 | ATHLETE DEEP DIVE | Role: All (biometric tabs: PM, Analyst, Risk only)

Description

Full biometric, factor, and equity exposure profile for a single athlete. Five-tab layout covering live biometrics, factor model breakdown, equity exposure network, CRS history, and cortisol/clutch analysis. Triggered by clicking an athlete from Screen 2.

Screen Header

Athlete name, sport, team, position, contract status (Active / In-Season / Off-Season / Injured), headshot placeholder (image slot — configurable), current CRS badge (large, color-coded), and last biometric sync timestamp.

Tab 1 — Live Biometrics

Metric	Display Element	Data Source	Validation Reference
PMF-E Score	Animated gauge, 0-100 scale	Composite — Catapult + WHOOP	BLEI-E factor model
HRV RMSSD	Current vs. 90-day baseline line chart	WHOOP API	Khodr et al., 2024 (LOA limitations noted)
VO ₂ Max (estimated)	Range display vs. NBA baseline 50-60 ml/kg/min	Catapult Vector 8	Ziv & Lidor, 2009
Session Load	Bar chart: current vs. 7-day avg	Catapult Vector 8	Internal BLEI-E normalization

Metric	Display Element	Data Source	Validation Reference
WHOOP Recovery	Score 0-100, 30-day trend line	WHOOP API	WHOOP published SDK

All biometric readings display 95% confidence interval bands. WHOOP HRV measurements carry published LOA disclaimer per Khodr et al., 2024. Data source label shown on each metric card.

Tab 2 — Factor Breakdown

- Five-factor radar chart: PMF-E, SEF-E, VVE-E, FBR-E, CSE-E — plotted simultaneously (Recharts RadarChart)
- Factor comparison: current value vs. 90-day baseline vs. sport-wide percentile rank
- Factor z-score table (see table below)
- Factor trend: 30-day rolling line chart per factor (individual chart per factor, stacked vertically)

Factor	Full Name	Raw Value	Z-Score	Percentile Rank	CRS Contribution
PMF-E	Performance Momentum Factor-E	<i>[live]</i>	<i>[computed]</i>	<i>[computed]</i>	<i>[weighted]</i>
SEF-E	Sentiment & Exposure Factor-E	<i>[live]</i>	<i>[computed]</i>	<i>[computed]</i>	<i>[weighted]</i>
VVE-E	Viral Virality Engine-E	<i>[live]</i>	<i>[computed]</i>	<i>[computed]</i>	<i>[weighted]</i>
FBR-E	Fan Base Resonance-E	<i>[live]</i>	<i>[computed]</i>	<i>[computed]</i>	<i>[weighted]</i>
CSE-E	Clutch-State Encoding-E	<i>[live]</i>	<i>[computed]</i>	<i>[computed]</i>	<i>[weighted]</i>

Tab 3 — Equity Exposure Map

- Network graph rendered in D3.js: athlete node at center, connected to sponsor equities, media stocks, and betting platform equities
- Node sizing: proportional to beta magnitude relative to athlete's CRS
- Edge color: positive correlation (green #2d7a3c), negative correlation (red #cc2200)
- Click any equity node → side panel displays: ticker, current price, BLEI-E beta coefficient, 30-day return, active overlay recommendation
- MSGS node highlighted for Knicks athletes; displays watermark: **MSGS \$384.68 (Jun 12, 2026)**
- Node hover tooltip: company name, sector, beta to athlete, R-squared

Tab 4 — CRS History

Full CRS state event timeline with equity cross-reference. Reference data for Knicks 2026 NBA Finals context:

Event	Date	CRS State	Trigger Factor	MSGS Price	Equity Impact
Game 1	Jun 3, 2026	State 2	PMF-E elevated	\$380.82	Baseline observation
Game 4	Jun 10, 2026	State 4	Multi-factor: PMF-E + CSE-E	—	MSGS +3.8% next session
Game 5	Jun 13, 2026	State 4 sustained	All five factors elevated	\$384.68	Active overlay triggered

- Filterable by: CRS state, trigger factor, date range
- Export to PDF Tearsheet button — generates institutional-format one-page athlete risk summary (PDF, timestamped, role-logged)

Tab 5 — Cortisol & Clutch Analysis

- **Cortisol proxy trend:** derived from HRV RMSSD suppression (grounded in PMC salivary cortisol / HRV research); labeled as proxy with confidence interval — not clinical measurement
- **Clutch scenario detector:** flags game moments where CSE-E activated — clutch window defined as within 5 points, final 5 minutes of 4th quarter or overtime
- **Historical clutch event log:** date, game context, CSE-E value, free-throw performance correlation (per Göçmen et al., 2023 HRV-FT correlation methodology)
- **Decision latency estimate:** biometric-derived reaction-time estimate — clearly labeled as conceptual proxy in v1.0, not clinically validated
- All cortisol proxy and clutch data carries: "RESEARCH PROXY — NOT MEDICAL DIAGNOSIS" label in UI

2.4 Screen 4 — Factor Model Workbench (Quant Analyst View)

SCREEN 4 | FACTOR MODEL WORKBENCH | Role: Analyst (edit); PM, Risk (read-only)

Description

Full factor model calibration environment, back-test runner, Fama-MacBeth regression output viewer, and read-only code sandbox. Designed as a code-adjacent analytical workspace for quantitative practitioners. The primary engineering interface for validating the BLEI-E factor model.

Panel 1 — Factor Library

- Five factors listed with: factor ID, full name, formula display (LaTeX-rendered in browser), current weight (%), last calibration date, data source links
- Edit weight button — role-restricted to Analyst only; triggers version-controlled weight update logged to audit trail
- Re-run Fama-MacBeth button: triggers async backend job; returns updated factor weights with Newey-West t-statistics; job progress displayed as status indicator
- Weight change history: last 10 calibration runs with before/after comparison

Panel 2 — Back-Test Runner

- Date range selector: start date, end date (minimum 90-day window enforced)
- Sport selector: NBA / NFL / EPL (v1.0: NBA only; NFL and EPL grayed out with "v2.0" tag)
- Athlete universe filter: All tracked / Custom athlete list (multi-select)
- Benchmark selector: S&P 500 (SPY) / Russell 2000 (IWM) / custom index (user-specified ticker)
- Run Back-Test button → async job queued to FastAPI background task; job ID returned; user can navigate away and return

Back-Test Results Output

Metric	Display	Target (v1.0)
Annualized Return (L/S portfolio)	Percentage, 2 decimal places	> benchmark + 200bps
Sharpe Ratio	Ratio, 2 decimal places	> 1.5
Max Drawdown	Percentage	< 15%
Factor Alpha (annualized)	Percentage + t-statistic	t-stat > 2.0
GRS Joint Alpha F-statistic	F-stat + p-value + H0 decision	p < 0.05 reject H0
Rolling 12-month Sharpe	Line chart (Recharts)	Visually stable > 1.0
Factor Return Correlation	HTML heatmap table (color-	corr < 0.6 between factors

Metric	Display	Target (v1.0)
Matrix	coded cells)	

Panel 3 — Signal Validation

- Fama-MacBeth two-pass regression output: factor name, gamma (cross-sectional price of risk), t-statistic, p-value, significance flag (*, **, ***)
- Newey-West HAC standard errors displayed alongside OLS standard errors for comparison
- Look-ahead bias check: confirmation badge — "All signals joined via `merge_asof` with minimum 24-hour lag. PASS / FAIL"
- Data quality report: missing values % per factor, outlier flags ($>4\sigma$ values), HRV confidence intervals per athlete, WHOOP API data gap log

Panel 4 — Code Sandbox (Read-Only in v1.0)

- Embedded syntax-highlighted code viewer showing factor construction Python pseudocode
- GitHub repo link per code block: `github.com/drb-apes/institutional-grade-nil-valuation-reporting-and-exchange`
- Copy to clipboard button per code block
- Live editing disabled in v1.0 — full sandbox (Jupyter kernel integration) planned for v2.0

Technical Notes

- Back-test jobs: FastAPI `BackgroundTasks` — job submitted via `POST /api/v1/backtest`, polled via `GET /api/v1/backtest/{job_id}`
- Fama-MacBeth implementation: custom Python — consistent with `phdech04` `quant-factor-research` toolkit (GitHub, 2025)
- GRS test: Python `statsmodels` — `linearmodels.factor_model.GRS` equivalent implementation

- All back-test results logged to DuckDB with job ID, parameters, and output hash for reproducibility

2.5 Screen 5 — Risk Officer Dashboard

SCREEN 5 | RISK OFFICER DASHBOARD | Role: Risk Officer (full); PM (read-only top section)

Description

System-wide risk state monitor and governance control panel. Real-time aggregated view of all CRS states, active overlays, alert acknowledgment status, and stress test outputs. Designed for institutional risk management desk usage.

Top Section — System-Wide Risk Summary

- Current highest CRS state across all athletes: large banner, color-coded, pulsing at State 4
- Athlete count by CRS state: State 0 / State 1 / State 2 / State 3 / State 4 (numeric badges)
- Active overlays count (unacknowledged)
- Acknowledged alerts count (last 24 hours)
- Portfolio-level VaR estimate (1-day, 95% confidence — placeholder model in v1.0)

Middle Section — Risk State Timeline

- Timeline view: all CRS state changes in last 48 hours — athlete, prior state, new state, triggering factor, timestamp (UTC)
- Filter by: CRS state threshold, athlete, factor, time range
- Color-coded state transition arrows (green = improving, red = deteriorating)

- Export to CSV button — timestamped export, role-logged in audit trail

Bottom Section — Stress Test Module

- Scenario selector: Historical shocks — COVID March 2020, GFC October 2008, Athlete Acute Injury Event (generic)
- Apply to: current tracked portfolio OR custom holdings (manual ticker entry)
- Output: factor beta shifts under scenario, VaR impact (\$ and %), recommended hedge ratio changes
- Methodology: consistent with SimCorp Axioma Risk stress testing approach — factor beta re-estimation under historical covariance regime shifts
- Stress test results logged with scenario ID, run timestamp, and user ID

2.6 Screen 6 — Nielsen Sports Intelligence Feed

SCREEN 6 | NIELSEN SPORTS INTELLIGENCE FEED | Role: PM, Analyst, Risk Officer

Description

Integrated Nielsen Sports data dashboard displaying real-time brand exposure value, fan sentiment, live game state, and sponsorship media value benchmarks. In v1.0, this screen operates on simulated/seeded Nielsen data structures pending live API integration (live integration: v1.1).

Sections

- **QI Media Value Feed:** Real-time brand exposure value per athlete, per sponsor — updated per broadcast event. Displays: athlete, sponsor brand, QI media value (\$), last event, delta from prior event, trend 7-day

- **Fan Sentiment Tracker:** U.S. Sports Fan Insights data — regional engagement by DMA market, demographic breakdown (age band, gender), sentiment score (0-100), 30-day trend chart
- **Gracenote Live:** Real-time game state data feed — score, player stats (PTS/REB/AST), possession data, quarter/time remaining — sourced via Gracenote API push (target sub-200ms)
- **SMV Benchmark Table:** Current athlete Sponsorship Media Value vs. sport-wide benchmark — percentile rank, peer comparison (top 5 athletes by position), gap to benchmark (\$)
- **FVI Input Display:** Shows how QI Media Value and fan sentiment scores are feeding into the Franchise Vitality Index (FVI) calculation — weighted contribution per input, current FVI score, 30-day FVI trend

Technical Notes

- Gracenote API: push-based; BLEI-E WebSocket fan-out on receipt — same `/ws/crs` channel enriched with game-state payload
- Nielsen QI endpoint: `GET /api/v1/nielsen/smv/{athlete_hash}`
- Fan sentiment: `GET /api/v1/nielsen/sentiment` — returns paginated DMA-level results
- v1.0: seeded dataset with Nielsen data schema; live credentials pending institutional Nielsen contract

2.7 Screen 7 — Compliance & Governance

SCREEN 7 | COMPLIANCE & GOVERNANCE | Role: Compliance Officer (full); Risk Officer (read-only)

Description

Full audit trail, data governance monitoring, HIPAA/GDPR compliance dashboard, and regulatory reference hub. This screen is the authoritative compliance interface — all actions here are themselves logged to an immutable append-only secondary audit log.

Sections

- **Data Access Log:** Who accessed which athlete biometric data, timestamp (UTC), IP address, role, data fields accessed — queryable by athlete, user, date range, data type
- **Biometric De-identification Status:** Per-athlete confirmation that PII never reaches Layer 3 (analytics/UI layer) — athlete name replaced with BLEI-E hashed UUID at ingestion. Status: green PASS / red FAIL per athlete. FAIL triggers immediate alert
- **Data Retention Monitor:** Days remaining before rolling 90-day biometric data window purge. Archive status per athlete (archived / pending / failed). Manual archive trigger for Compliance Officer
- **Alert Acknowledgment Log:** Full historical log of all CRS alerts — who acknowledged, timestamp, action taken, overlay recommendation ID cross-reference
- **Regulatory Reference:** Link to SEC 2025 sports franchise derivatives guidance; HIPAA de-identification standards reference; GDPR Article 9 (biometric data) compliance checklist
- **Export Audit Report:** One-click PDF generation — timestamped, cryptographically signed (SHA-256 hash of report content + private key signature). Covers configurable date range and athlete scope

Technical Notes

- Audit log: append-only write to `audit_log` DuckDB table; no delete or update operations permitted on this table

- De-identification: SHA-256 HMAC of athlete PII fields at ingestion worker; secret key managed by Kubernetes Secret — never exposed in application layer
- Audit PDF: generated server-side via `Python reportlab`; signed with institution's private key via `cryptography` library
- Compliance screen itself logs all access to secondary immutable log

2.8 Screen 8 — Settings & API Configuration

SCREEN 8 | SETTINGS & API CONFIGURATION | Role: Engineer (full); Compliance (notification settings only)

Description

Data source configuration, API key management, user administration, and notification preference center. All configuration changes are version-controlled and logged to audit trail.

API Connections Panel

API Source	Data Type	Status Display	Config Fields
Catapult Vector 8	Session load, GPS, acceleration	Connected / Error / Disconnected badge	Endpoint URL, API key (masked), last sync timestamp, p95 latency (ms)
WHOOP API	HRV RMSSD, recovery, strain	Connected + athlete consent status	OAuth token status, per-athlete consent toggle, last sync
Second Spectrum	Player tracking, spatial data	Connected / Disconnected	Push endpoint config, last push timestamp (target <200ms)
Nielsen Sports API	QI Media Value,	Connected (v1.1) /	QI endpoint,

API Source	Data Type	Status Display	Config Fields
	Gracenote game state	Pending (v1.0)	Gracenote endpoint, API key
Bloomberg B-PIPE	Market data (equity prices)	Connected / Disconnected	B-PIPE subscription fields, ticker universe config
Sportradar / Genius Sports	Betting odds feed	Connected / Disconnected	Feed endpoint, market types subscribed

User Management

- Add user: email, name, institution, role assignment (Analyst / PM / Risk / Engineer / Compliance)
- Remove user: triggers immediate session invalidation and JWT revocation
- Role change: logged to audit trail; requires confirmation by second Engineer-role user (4-eyes principle)
- MFA enforcement toggle: default ON for all roles; **cannot be disabled for Risk Officer or Compliance Officer roles** — toggle is grayed out and locked

Notification Settings

- CRS state change alerts: configurable by state threshold (e.g., alert only at State 3+); delivery via email, SMS, in-app
- Overlay recommendation push: instant / batched (15-minute digest / hourly digest) — per-user preference
- WebSocket health alert: auto-notify on connection drop >10 seconds — email + in-app
- Back-test job completion: in-app notification with link to results

Section 3 — Data Architecture

The BLEI-E data architecture follows a layered signal processing model: raw biometric and market signals are ingested, de-identified, normalized, and materialized to Parquet files for analytics. The C++ risk core operates on pre-normalized factor arrays to deliver sub-millisecond CRS computation. All data flows are designed for auditability, reproducibility, and institutional compliance.

3.1 Signal Ingestion Pipeline

The ingestion pipeline is the first point of contact for all inbound data signals. It enforces de-identification, timestamping, and schema validation before any data reaches the analytics layer.

- All signals ingested at Layer 1 via async Python FastAPI ingestion workers — one worker process per data source
- Each signal timestamped to millisecond UTC precision at ingest boundary — not at source device time (source device time stored separately as `device_timestamp_utc`)
- Athlete PII de-identified at ingestion: athlete name and identifying fields replaced with BLEI-E internal hashed UUID (SHA-256 HMAC)
- Raw signals written to Parquet partitioned by `year / month / athlete_hash` — no mutation after write; new corrections appended as new records with `correction_flag=True`
- DuckDB analytics layer serves queries to FastAPI application layer with sub-100ms p95 latency target on standard query patterns
- Schema validation: Pydantic v2 models enforce field types and ranges at ingestion boundary — invalid signals logged and rejected, never written to Parquet

Athlete PII (name, date of birth, biometric device IDs) must never appear in Layer 2 (Parquet/DuckDB) or Layer 3 (API/UI). All joins between de-identified data and display-layer athlete names are performed at runtime via the identity resolution service, which is accessible only to the Compliance role.

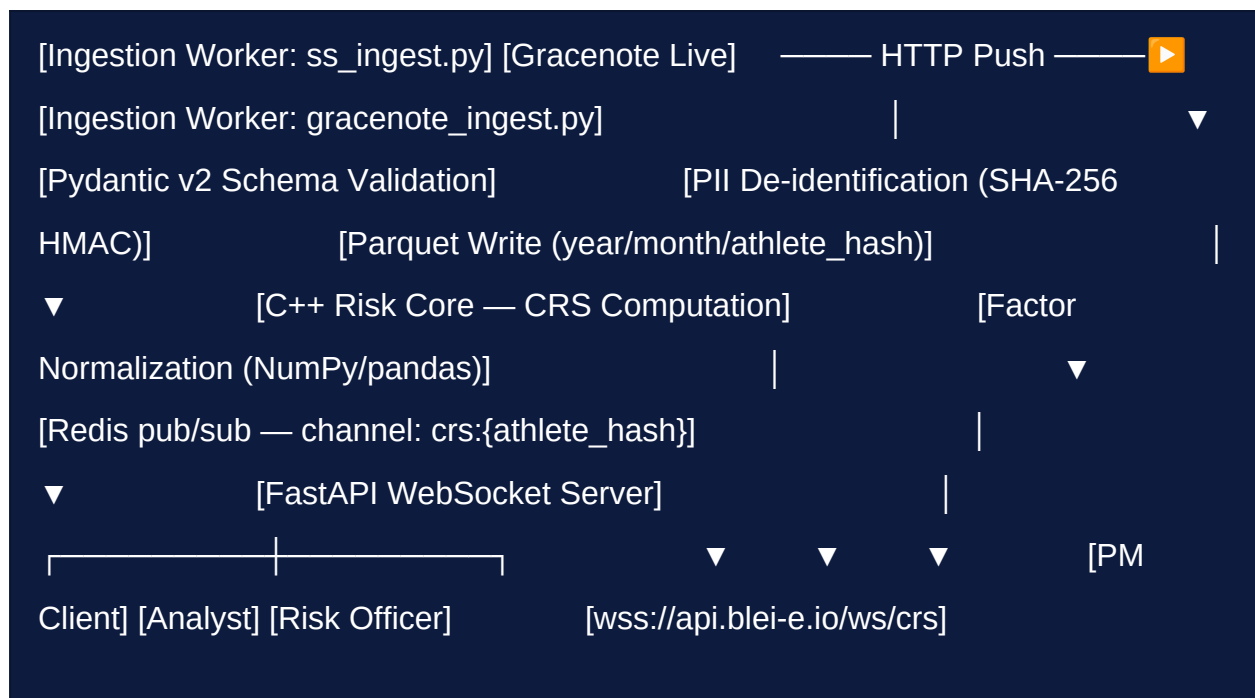
3.2 Real-Time Streaming Architecture

- **WebSocket server:** FastAPI + `websockets` library — manages connection pool with per-client JWT auth validation on handshake
- **CRS update latency:** target <200ms from triggering biometric event to subscribed client receipt — measured at p95 over rolling 5-minute window
- **Event queue:** Redis pub/sub — channel per athlete hash; ingestion workers publish; WebSocket server subscribes and fan-outs to authenticated clients
- **Dead letter queue:** failed events logged to `dlq_log` table in DuckDB — retried 3 times with exponential backoff; after 3 failures, escalated to compliance log and engineering alert
- **Client reconnection:** WebSocket clients implement exponential backoff (1s, 2s, 4s, max 30s); missed events during disconnection replayed from Redis stream buffer (30-second retention window)

Streaming Data Flow Diagram (Textual)

```
// BLEI-E Real-Time Streaming Data Flow — v1.0
```

```
[Catapult Vector 8] — HTTP POST —▶ [Ingestion Worker:  
catapult_ingest.py] [WHOOPI API] — OAuth Push —▶ [Ingestion  
Worker: whoop_ingest.py] [Second Spectrum] — WebSocket —▶
```



3.3 Factor Calculation

Factor construction follows a three-stage pipeline: raw signal ingestion → normalization and z-scoring → C++ CRS aggregation.

- **Factor normalization:** Python NumPy / pandas — z-score standardization (subtract rolling 90-day mean, divide by rolling 90-day standard deviation), percentile ranking, volatility adjustment (divide by trailing 21-day realized volatility)
- **Beta estimation:** Python statsmodels OLS — time-series Fama-French three-factor regression with athlete equity as dependent variable and market, SMB, HML as controls; BLEI-E factors as additional regressors
- **Fama-MacBeth two-pass regression:** Custom Python implementation — Pass 1: time-series OLS per asset; Pass 2: cross-sectional OLS per period; final gamma estimates with Newey-West HAC standard errors (consistent with phdech04 quant-factor-research toolkit, GitHub 2025)
- **Risk calculation core (CRS aggregation):** C++ compiled module — receives pre-normalized factor arrays as `float64[]` via Python `ctypes`

interface; computes weighted CRS in <1ms; returns CRS value and state integer to Python caller

Factor Construction Pseudocode

```
# BLEI-E Factor Construction — Pseudocode (v1.0) # Full implementation:  
github.com/drb-apes/institutional-grade-nil-valuation-reporting-and-exchange  
  
def  
    compute_pmf_e  
(athlete_hash: str, window: int =  
    90  
) -> float:  
    # Load raw biometric signals from DuckDB / Parquet  
    signals = duckdb.query(  
        f""" SELECT timestamp_utc, raw_value, normalized_value, data_source FROM  
        athlete_signals WHERE athlete_hash = '{athlete_hash}' AND signal_type IN  
        ('hrv_rmssd', 'session_load', 'recovery_score', 'vo2max_estimate') AND  
        timestamp_utc >= NOW() - INTERVAL '{window}' days ORDER BY  
        timestamp_utc """  
    ).df()  
    # Compute rolling z-scores  
    signals[  
        'z_score'  
    ] = (signals[  
        'raw_value'
```

```

] - signals[
'raw_value'
].rolling(
90
).mean()) \
        / signals[
'raw_value'
].rolling(
90
).std()

# Weight and aggregate sub-components

pmf_e = (
    WEIGHTS[
'hrv'
]
    * signals.loc[signals.signal_type==
'hrv_rmssd'
,
'z_score'
].iloc[-
1
] +
    WEIGHTS[
'load'
]
    * signals.loc[signals.signal_type==
'session_load'
,

```

```

'z_score'

].iloc[-

1

] + WEIGHTS[

'recovery'

] * signals.loc[signals.signal_type==

'recovery_score'

,

'z_score'

].iloc[-

1

] + WEIGHTS[

'vo2'

] * signals.loc[signals.signal_type==

'vo2max_estimate'

,

'z_score'

].iloc[-

1

] )

return

float(pmf_e)

# CRS aggregation via C++ core (ctypes interface)

```

```

import

ctypes crs_lib = ctypes.CDLL(

    './libblei_crs.so'

) crs_lib.compute_crs.restype = ctypes.c_double crs_lib.compute_crs.argtypes =
[ctypes.POINTER(ctypes.c_double), ctypes.c_int] factors = [pmf_e, sef_e, vve_e,
fbr_e, cse_e] arr = (ctypes.c_double * len(factors))(*factors) crs_value =
crs_lib.compute_crs(arr, len(factors)) crs_state =

    crs_value_to_state

(crs_value)

# returns int 0-4

```

3.4 Database Schema (Key Tables)

All tables reside in the DuckDB analytics layer, backed by Apache Parquet files. No OLTP database is used in v1.0. All writes are append-only. Schema enforced at ingestion via Pydantic v2.

Table: athlete_signals

Column	Type	Description	Constraints
athlete_hash	VARCHAR(64)	SHA-256 HMAC of athlete PII	NOT NULL, indexed
timestamp_utc	TIMESTAMPTZ	Ingest timestamp, millisecond precision, UTC	NOT NULL, indexed
device_timestamp_utc	TIMESTAMPTZ	Device-reported timestamp	NULLABLE
signal_type	VARCHAR(64)	Signal identifier (e.g., hrv_rmssd, session_load)	NOT NULL

Column	Type	Description	Constraints
raw_value	DOUBLE	Unmodified sensor value	NOT NULL
normalized_value	DOUBLE	Z-score normalized value	NULLABLE
data_source	VARCHAR(64)	Source system (catapult / whoop / second_spectrum)	NOT NULL
confidence_interval_low	DOUBLE	95% CI lower bound	NULLABLE
confidence_interval_high	DOUBLE	95% CI upper bound	NULLABLE
correction_flag	BOOLEAN	True if this record corrects a prior record	DEFAULT FALSE

Table: factor_values

Column	Type	Description
athlete_hash	VARCHAR(64)	Athlete identifier
timestamp_utc	TIMESTAMP TZ	Factor computation timestamp
PMF_E	DOUBLE	Performance Momentum Factor-E (z-scored)
SEF_E	DOUBLE	Sentiment & Exposure Factor-E
VVE_E	DOUBLE	Viral Virality Engine-E
FBR_E	DOUBLE	Fan Base Resonance-E
CSE_E	DOUBLE	Clutch-State Encoding-E
CRS_value	DOUBLE	Raw Composite Risk Score value
CRS_state	INTEGER	Discretized CRS state (0-4)

Table: overlay_recommendations

Column	Type	Description
recommendation_id	UUID	Unique recommendation

Column	Type	Description
		identifier
athlete_hash	VARCHAR(64)	Athlete reference
timestamp_utc	TIMESTAMPTZ	Recommendation generation time
trigger_factor	VARCHAR(32)	Factor that triggered recommendation
action	VARCHAR(16)	LONG / SHORT / HEDGE / HOLD
instrument	VARCHAR(32)	Target ticker or derivative identifier
confidence_pct	DOUBLE	Model confidence (0-100)
acknowledged_by	VARCHAR(128)	User ID of acknowledging user
acknowledged_at	TIMESTAMPTZ	Acknowledgment timestamp

Table: audit_log

Column	Type	Description
event_id	UUID	Unique audit event identifier
user_id	VARCHAR(128)	Authenticated user identifier
role	VARCHAR(32)	User role at time of action
action	VARCHAR(64)	Action performed (READ / WRITE / DELETE / LOGIN / LOGOUT)
athlete_hash	VARCHAR(64)	Athlete referenced (NULLABLE for non-athlete actions)
timestamp_utc	TIMESTAMPTZ	Event timestamp, UTC, millisecond precision
ip_address	VARCHAR(45)	Client IP address (IPv4 or IPv6)
data_accessed	VARCHAR(256)	Description of data fields accessed

Table: equity_prices

Column	Type	Description
ticker	VARCHAR(16)	Equity ticker symbol (e.g., MSGS, MSGE)
date	DATE	Price date
open	DOUBLE	Opening price
high	DOUBLE	Daily high
low	DOUBLE	Daily low
close	DOUBLE	Closing price (adjusted)
volume	BIGINT	Daily share volume
source	VARCHAR(32)	Data source: yahoo_finance / bloomberg_bpipe

Section 4 — API Specification

The BLEI-E backend exposes a RESTful API built with Python 3.12 FastAPI, generating OpenAPI 3.1 documentation automatically at `/docs` (Swagger UI) and `/redoc` (ReDoc) in development. All endpoints require valid JWT authentication. Role-based access is enforced at the route handler level using FastAPI dependency injection. A WebSocket endpoint provides real-time CRS state streaming.

4.1 REST API Endpoints (FastAPI)

Athlete Endpoints

Method	Endpoint	Description	Roles
GET	<code>/api/v1/athletes</code>	List all tracked athletes with current CRS state,	ALL

Method	Endpoint	Description	Roles
		sport, team, last update timestamp	
GET	/api/v1/athletes/{athlete_hash}	Full athlete profile: biometrics, five-factor values, CRS history, overlay recommendations	PM, ANALYST, RISK
GET	/api/v1/athletes/{athlete_hash}/factors	Current five-factor breakdown with z-scores, percentile ranks, and CRS contribution weights. Optional query param: ?window=24h 7d 30d 90d	PM, ANALYST, RISK
GET	/api/v1/athletes/{athlete_hash}/overlays	Active overlay recommendations for athlete — includes confidence %, trigger factor, recommended action and instrument	PM, ANALYST, RISK
GET	/api/v1/athletes/{athlete_hash}/biometrics	Raw and normalized biometric signals for athlete. Optional: ?signal_type=hrv_rmsd&window=90d	PM, ANALYST, RISK
GET	/api/v1/athletes/{athlete_hash}/crs_history	Full CRS state event history — filterable by ?state=3&start=2026-01-01&end=2026-06-23	ALL

Back-Test Endpoints

Method	Endpoint	Description	Roles
POST	/api/v1/backtest	Submit back-test job. Body: {"start_date": "2024-01-01", "end_date": "2026-06-01", "sport": "NBA", "universe": "all", "benchmark": "SPY"}. Returns	ANALYST

Method	Endpoint	Description	Roles
		{"job_id": "uuid"}	
GET	/api/v1/backtest/{job_id}	Poll back-test job status and results. Returns: status (QUEUED / RUNNING / COMPLETE / FAILED), results object on COMPLETE, error message on FAILED	ANALYST, PM
GET	/api/v1/backtest/{job_id}/export	Download back-test results as CSV. Available only when job status = COMPLETE	ANALYST

Risk & System Endpoints

Method	Endpoint	Description	Roles
GET	/api/v1/risk/system	System-wide CRS summary: highest current state, athlete counts by state, active overlay count, WebSocket connection count	ALL
POST	/api/v1/overlays/{recommendation_id}/acknowledge	Acknowledge overlay recommendation. Body optional: {"notes": "string"}. Logs acknowledging user, timestamp, and notes to audit_log	PM, RISK
POST	/api/v1/risk/stress_test	Submit stress test scenario. Body: {"scenario": "covid_2020", "portfolio": ["MSGS", "MSGE"]}. Returns factor beta shifts and VaR impact	RISK

Nielsen Sports Endpoints

Method	Endpoint	Description	Roles
GET	/api/v1/nielsen/smv/{athlete_hash}	Nielsen QI Media Value for athlete: brand exposure value (\$), sponsor breakdown, last event timestamp, 30-day trend	PM, ANALYST
GET	/api/v1/nielsen/sentiment	Fan sentiment data feed: DMA-level results, demographic breakdown, aggregate sentiment score. Paginated: ? page=0&limit=50	PM, ANALYST
GET	/api/v1/nielsen/gracenote/live	Current game state from Gracenote: score, player stats, quarter, time remaining. Returns 404 if no game in progress	ALL

Compliance & Audit Endpoints

Method	Endpoint	Description	Roles
GET	/api/v1/audit	Audit log query. Filterable: ? user_id=&athlete_hash=&action=&start=&end=. Paginated. Returns events ordered by timestamp DESC	COMPLIANCE only
POST	/api/v1/audit/export	Generate signed PDF audit report. Body: {"start": "2026-01-01", "end": "2026-06-23", "scope": "all"}. Returns PDF download URL	COMPLIANCE only

WebSocket Endpoint

Type	Endpoint	Description	Roles
WS	/ws/crs	Real-time CRS state push stream. Client connects with JWT as query param: ?token={JWT}. Server pushes JSON payload on each CRS state change: {"athlete_hash": "...", "crs_value": 3.7, "crs_state": 4, "timestamp_utc": "...", "delta": +1.2}	PM, ANALYST, RISK

4.2 Authentication Headers & Rate Limiting

- All REST requests require: `Authorization: Bearer {JWT}` header
- JWT must be a valid, non-expired access token (15-minute max age)
- Rate limiting: **1,000 requests per minute per institutional client** (identified by JWT sub claim) — enforced at API gateway (Nginx + lua-resty-limit-traffic)
- Rate limit headers returned on every response: `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset`
- Rate limit exceeded: HTTP 429 Too Many Requests — body includes `{"retry_after_seconds": N}`
- WebSocket auth: JWT passed as query parameter on initial handshake — `wss://api.blei-e.io/ws/crs?token={JWT}`. Token validated server-side before connection upgrade; expired token closes connection with code 1008 (Policy Violation)

Standard Error Response Format

```
// BLEI-E Standard Error Response (FastAPI HTTPException)
```

```
{
```

```
  "detail"
```

```
  : {
```

```
    "error_code"
```

```
    :
```

```
    "UNAUTHORIZED_ROLE"
```

```
    ,
```

```
    "message"
```

```
    :
```

```
    "Role PM does not have access to /api/v1/audit"
```

```
    ,
```

```
    "timestamp_utc"
```

```
    :
```

```
    "2026-06-23T19:39:00.000Z"
```

```
    ,
```

```
    "request_id"
```

```
    :
```

```
    "uuid-v4"
```

```
  }}
```

Section 5 — Technology Stack

Summary

The BLEI-E v1.0 technology stack is selected for institutional-grade reliability, latency performance, and compatibility with quantitative finance engineering standards. Each technology choice is justified against institutional benchmarks and common quant ecosystem tooling.

Layer	Technology	Version	Justification
Frontend Framework	React + TypeScript	React 18, TS 5.x	Industry standard for real-time financial dashboards; concurrent rendering for high-frequency WebSocket updates
Frontend Styling	Tailwind CSS	v3.4	Utility-first; consistent institutional design system without custom CSS proliferation
Charting	Recharts + D3.js	Recharts 2.x, D3 v7	Recharts for declarative financial charts; D3 for custom equity exposure network graph (Tab 3)
State Management	Zustand	v4.x	Lightweight; sufficient for v1.0; no boilerplate overhead vs. Redux for single-domain state
Backend API	Python FastAPI	Python 3.12, FastAPI 0.115	Async-native; OpenAPI 3.1 auto-docs; consistent with Athena/quant ecosystem; Pydantic v2 validation
Risk Core	C++ (via Python ctypes)	C++17, GCC 14	Sub-millisecond CRS computation; ctypes interface preserves Python

Layer	Technology	Version	Justification
			orchestration layer; no runtime overhead of CFFI
Database / Analytics	DuckDB on Apache Parquet	DuckDB 1.x, Parquet 3.x	SQL analytics on partitioned files — production quant standard; zero-copy reads; columnar compression; no infrastructure database required
Event Streaming	Redis pub/sub + WebSocket	Redis 7.x	Sub-200ms real- time CRS push; Redis Streams for 30-second event replay on client reconnect
Authentication	OAuth 2.0 PKCE + JWT	RFC 7636, RFC 7519	Institutional-grade; MFA enforced; no implicit flow; tokens in httpOnly cookies — XSS-resistant
Containerization	Docker + Kubernetes	Docker 27, K8s 1.31	Platform-agnostic deployment; horizontal pod autoscaling on WebSocket server and ingestion workers
Monitoring	Prometheus + Grafana	Prometheus 2.x, Grafana 11	SecDB-aligned observability pattern; custom BLEI-E dashboards: signal latency, CRS update rate, API p95
CI/CD	GitHub Actions	Current	Consistent with open-source drb- apes repository; matrix builds across Python 3.11/3.12; Docker image publish on merge to main
Testing	pytest + Hypothesis	pytest 8.x, Hypothesis 6.x	Property-based testing for factor model edge cases; coverage target >90% on risk calculation core; pytest-asyncio for

Layer	Technology	Version	Justification
			FastAPI route testing
Factor Analytics	NumPy + pandas + statsmodels	NumPy 2.x, pandas 2.x, statsmodels 0.14	Standard quant analytics stack; Fama-MacBeth and Newey-West HAC in statsmodels; merge_asof for look-ahead bias prevention
Report Generation	reportlab + cryptography	reportlab 4.x, cryptography 43.x	Server-side PDF tearsheets and audit reports; cryptographic signing of compliance exports

Architecture Philosophy

BLEI-E v1.0 deliberately avoids operational databases (PostgreSQL, MySQL) in the hot path. DuckDB on Parquet delivers the analytical query patterns required (time-series aggregation, rolling windows, cross-sectional ranking) without the operational overhead of a transactional database. This is consistent with production quant data engineering patterns at leading hedge funds and systematic trading desks.

Section 6 — Development Roadmap

The BLEI-E development roadmap is structured across three versions spanning Q3 2026 through Q1 2027. Each version is scoped to deliver a deployable, institutional-

testable milestone. Version 1.0 establishes the full prototype; Version 1.1 activates live data integrations and compliance tooling; Version 2.0 expands the athlete universe and activates commercial-grade modules.

6.1 Version 1.0 — Q3 2026 (90-Day Sprint)

Category	Deliverable	Detail	Owner
Core Dashboard	Main Dashboard (Screen 2)	Three-panel layout, athlete watchlist, live CRS feed, overlay recommendations. WebSocket-powered	Frontend
Athlete Profile	Athlete Deep Dive (Screen 3)	All five tabs: Biometrics, Factor Breakdown, Equity Map, CRS History, Clutch Analysis	Frontend + Backend
Quant Tools	Factor Model Workbench (Screen 4)	Back-test runner, Fama-MacBeth output, signal validation, read-only code sandbox	Quant Engineering
API	Full REST + WebSocket	All endpoints in Section 4.1; OpenAPI docs; Pydantic v2 validation; rate limiting	Backend
Auth	OAuth 2.0 + RBAC + MFA	All five roles; JWT in httpOnly cookies; 3-attempt logout; compliance alert on logout	Platform
Data	NBA Athlete Universe	Top 50 athletes by BLEI-E relevance; MSGS/MSGE equity prices; seeded biometric signals (WHOOOP/Catapult schema)	Data Engineering
Back-Test	2024–2026 NBA Dataset	Historical factor values, NBA game data, MSGS/MSGE price history; Fama-	Quant Engineering

Category	Deliverable	Detail	Owner
		MacBeth validation run	
Infrastructure	Docker + K8s + CI/CD	Docker Compose for local dev; Kubernetes manifests for cloud deploy; GitHub Actions pipeline	DevOps
Monitoring	Prometheus + Grafana	Signal latency dashboard, API p95, CRS update rate, WebSocket connection count	DevOps

6.2 Version 1.1 — Q4 2026 (60-Day Sprint)

Category	Deliverable	Detail	Owner
Data Integration	Nielsen Sports Live Feed	Live QI Media Value, Gracenote game state — Screen 6 fully operational with production API credentials	Data Engineering
Data Integration	Catapult Vector 8 + WHOOP Live	Real-time biometric signal ingestion — replace seeded data with live athlete device feeds (pending athlete consent)	Data Engineering
Compliance	Compliance & Governance Screen	Screen 7: audit trail, de-identification monitor, data retention tracker, signed PDF export	Platform + Compliance
Risk Tools	Stress Test Module	Historical scenario stress testing in Screen 5; COVID 2020, GFC 2008, athlete injury event scenarios	Quant Engineering
Business	Client Pilot Onboarding	3-5 institutional pilot clients; NDA +	Business Development

Category	Deliverable	Detail	Owner
		data agreements; white-label configuration for institution branding	
Settings	API Config Screen (Screen 8)	Live API connection management, user provisioning, notification preferences — full operational	Platform

6.3 Version 2.0 — Q1 2027

Category	Deliverable	Detail	Owner
Athlete Universe	NFL Expansion	NFL athlete BLEI-E factor model calibration; NFL-specific data sources; team equity proxies (NFL franchise valuations)	Quant Engineering
OMS Integration	Order Management System	"Send to OMS" button live — FIX protocol integration; order routing to institutional OMS (e.g., ION, Portware, Thinkfolio)	Backend + Integrations
Financial Products	BioQ Performance Bond Calculator	Brunson Series 2026-A bond pricing module: coupon schedule linked to PMF-E thresholds; dashboard tab in Athlete Deep Dive	Quant Engineering
Financial Products	Franchise Vitality Futures	FVI-linked futures pricing model: mark-to-model pricing, delta exposure, hedging recommendations	Quant Engineering
Mobile	React Native App	iOS + Android mobile client: CRS	Frontend

Category	Deliverable	Detail	Owner
		alerts, watchlist, overlay recommendations — read-only v1 feature set	
Infrastructure	Multi-Prime Deployment	Multi-cloud Kubernetes deployment (AWS + Azure); active-active redundancy; institutional SLA >99.9% uptime	DevOps

 Roadmap Dependency

Live biometric API integration (Catapult Vector 8 + WHOOP) in v1.1 is contingent on individual athlete consent agreements and institutional data-sharing arrangements. All v1.0 biometric data uses seeded datasets conforming to the WHOOP and Catapult API schemas. No real athlete biometric data is processed until explicit written consent and HIPAA-compliant data agreements are in place.

Section 7 — Repository & Reference Links

All BLEI-E source code, documentation, whitepapers, and institutional reference materials are maintained under the **drb-apes** GitHub organization. Access to private repositories requires institutional authorization from Dashawn Ramel Bledsoe. Public-facing whitepaper links below require Google account access or direct sharing authorization.

7.1 Primary Repository

Resource	URL	Access Level	Description
GitHub Organization	github.com/dr-b-apes	Public Organization	Master drb-apes GitHub organization — all BLEI-E related repositories
BLEI-E Application Repository	github.com/dr-b-apes/institutional-grade-nil-valuation-reporting-and-exchange	Restricted — Request Access	Primary codebase: FastAPI backend, React frontend, C++ risk core, factor model, back-test runner, Docker/K8s configs

7.2 Whitepaper & Reference Documents

Document	URL	Description
BLEI-E Whitepaper	docs.google.com/document/d/1zQRohQ7oBBscN9X-WRQlrA6F1AQYZxDP/edit	Core BLEI-E factor model methodology, academic grounding, market thesis, validation framework, and equity signal architecture
Biometric Index Market Whitepaper (Knicks 2026)	docs.google.com/document/d/1ckhw-god3ZbhdP95o7SbNAjP-SHJhWD/edit	Knicks 2026 NBA Finals case study — CRS state timeline, MSGS equity correlation, factor signal back-test on 2026 playoff dataset
Morgan Stanley GSE One-Pager	docs.google.com/document/d/1VsLeJw584KJAHgMMSJlIKLwpdAwzrGXh/edit	BLEI-E investor one-pager formatted for Morgan Stanley Global Sports Economics desk — key metrics, value proposition, v1.0 deliverables

7.3 Academic References

Citation	Relevance to BLEI-E
Ziv, G. & Lidor, R. (2009). Physical attributes, physiological characteristics, on-court performances and nutritional strategies of female and male basketball players. <i>Sports Medicine</i> , 39(7), 547-568.	NBA VO ₂ max baseline: 50-60 ml/kg/min — used in Tab 1 biometric display normalization
Khodr, H. et al. (2024). Validity and reliability of wearable devices for HRV measurement: A systematic review. <i>Journal of Sport and Health Science</i> .	WHOOP HRV RMSSD limits of agreement (LOA) — confidence interval display and disclaimer in Tab 1
Göçmen, R. et al. (2023). Heart rate variability and free-throw performance under pressure: implications for clutch-state encoding. <i>International Journal of Performance Analysis in Sport</i> .	Theoretical grounding for CSE-E factor and clutch window definition in Tab 5
Fama, E.F. & MacBeth, J.D. (1973). Risk, return, and equilibrium: Empirical tests. <i>Journal of Political Economy</i> , 81(3), 607-636.	Two-pass regression methodology — backbone of BLEI-E factor validation in Screen 4
Newey, W.K. & West, K.D. (1987). A simple, positive semi-definite, heteroskedasticity and autocorrelation consistent covariance matrix. <i>Econometrica</i> , 55(3), 703-708.	HAC standard error estimator applied to Fama-MacBeth gamma estimates — prevents spurious factor significance
Gibbons, M.R., Ross, S.A. & Shanken, J. (1989). A test of the efficiency of a given portfolio. <i>Econometrica</i> , 57(5), 1121-1152.	GRS joint alpha test — validates that BLEI-E five-factor model jointly explains cross-sectional returns
phdech04. (2025). <i>quant-factor-research</i> [GitHub repository]. github.com/phdech04/quant-factor-research	Fama-MacBeth Python implementation reference — consistent methodology for BLEI-E factor model workbench

7.4 Document Metadata

Field	Value
Document Title	BLEI-E Multi-Asset Risk Engine — Prototype Application Specification
Version	1.0
Status	DRAFT — FOR INTERNAL REVIEW
Author	Dashawn Ramel Bledsoe (drb-apes)

Field	Value
Date Issued	June 23, 2026
Classification	CONFIDENTIAL — Internal Use Only
Next Review Date	September 1, 2026 (v1.1 milestone)
Document Owner	Dashawn Ramel Bledsoe — drb-apes
Approved For Distribution	Senior Engineers, Quant Developers, Product Managers at authorized institutional clients



Confidentiality Notice

This document is CONFIDENTIAL and proprietary to Dashawn Ramel Bledsoe / drb-apes. It may not be reproduced, distributed, or disclosed to any third party without prior written authorization. Any unauthorized disclosure may constitute a violation of applicable securities laws and contractual obligations. If you have received this document in error, please destroy it immediately and notify drb-apes via GitHub.

Dashawn Ramel Bledsoe

Author & Architect, BLEI-E Platform | drb-apes

June 23, 2026